

Automatique

Seite Alexander et Nathan Vidonne

April 5, 2026

Contents

1	Introduction	2
2	Exercice 1 : Analyse empirique du banc de test	2
2.1	A : Estimation de la fonction de transfert du système	3
2.2	B/C/D : Vérification de la fonction de transfert	3
3	Exercice 2 : Rentrions bien dans les détails	4
3.1	A : Verification graphique	5
3.2	B : Fonction de transfert du systeme complet	6
3.3	C : Code definissant la fonction de transfert	8
3.4	D : Application d'un regulateur PI	9
3.5	E : Diagrammes de Bode et Nyquist	10
3.6	F : Diagramme d'Evans et reponse indicielle	12
3.7	G : $G(s)$ dans "Control System Designer"	14
3.8	H : (MATLAB == CSD) ? verification graphique	16
3.9	I : Marges de stabilite sur diagramme de Bode	17
3.10	J : Lieu des poles (vous nous avez dit de passer cette question)	18
3.11	K : Caracteristiques de la reponse indicielle	19
3.12	L : Lien entre gain du regulateur et valeurs de depassement	21
3.13	M : Impact du gain sur la stabilite du systeme	24
4	Conclusion	27
5	annexes (code inexact à cause du compilateur latex)	28
5.1	Code complet matlab exercice 2	28
5.2	Code python point I	30
5.3	Code python point K	31
5.4	Code python point L	32

1 Introduction

Dans ce rapport, nous étudions le comportement d'un banc de test SIRAu. Il s'agit d'un motoreducteur instrumenté constitué donc d'un moteur à courant continu avec réducteur et génératrice tachymétrique. Ce dernier est piloté par un contrôleur de chez Maxon dont on ne dispose pas la référence du modèle.

Nous analysons en deux temps ce banc de test. D'abord, nous discutons par des représentations graphiques le comportement du banc de test à partir de données empiriques qui nous a permis de caractériser la fonction de transfert du modèle. Cette dernière nous permet en un second temps d'analyser la stabilité du système asservi. Nous comparerons les résultats représentés par les différentes méthodes de manière à être en mesure d'établir un lien entre elles.

Par conséquent, les objectifs sont de faire le lien entre un comportement mesuré sur une installation réelle et une représentation par fonction de transfert et de comprendre et expliciter ce que l'on peut observer sur les représentations graphiques usuelles (Bode, Nyquist, Evans. Ref. [?])

La complétion de ces objectifs permettront de nous familiariser avec les commandes MATLAB pour afficher ces graphs au travers l'utilisation de l'app "Control system designer et l'analyse des résultats.

2 Exercice 1 : Analyse empirique du banc de test



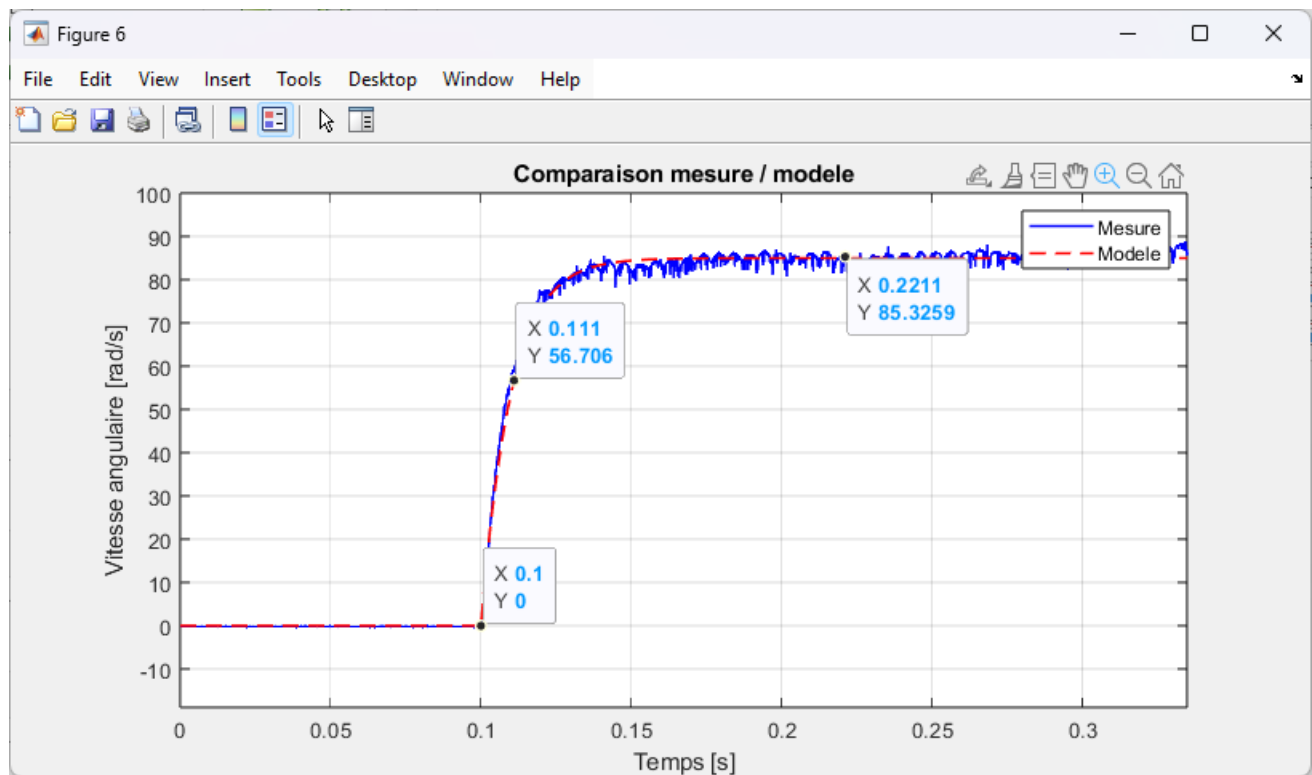
2.1 A : Estimation de la fonction de transfert du système

Pour le gain estime de $k = 85$ et une constante de temps $\tau = 0.01$, nous retrouvons un système du premier ordre :

$$G(s) = \frac{K}{1 + \tau \cdot s}$$

2.2 B/C/D : Vérification de la fonction de transfert

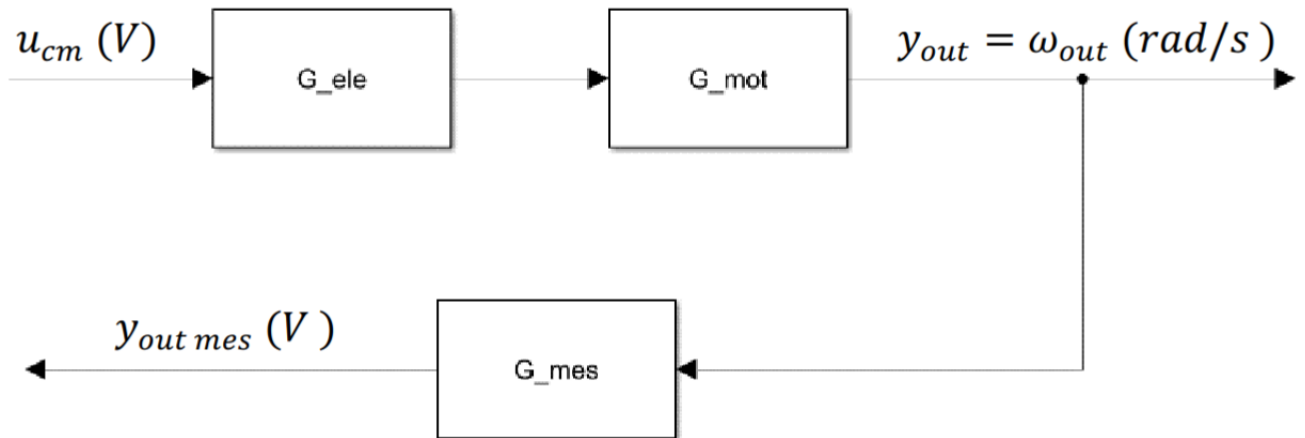
Dans le graphique suivant, nous pouvons observer que nous avons dû rajouter un retard de 0.1s afin de pouvoir comparer les données mesurées (bleu) et la courbe théorique (rouge).



Par définition, τ peut être déterminé à 66% du gain final. Nous trouvons 0.111. Cela n'est pas égal à la valeur théorique, mais cela s'explique par le fait que nous avons ajouté un retard de 0.1. Comme $0.111 - 0.1 = 0.011$, nous trouvons en effet une valeur proche du modèle pour τ . Le gain statique qu'illustre Fig. ?? est lui aussi proche de la valeur théorique ($85.3259 \approx 85$).

3 Exercice 2 : Rentrons bien dans les details

Nous schematisons notre systeme sur Fig. ?? par le produit de 3 sous-systemes (moteur, mesure, element) [?].



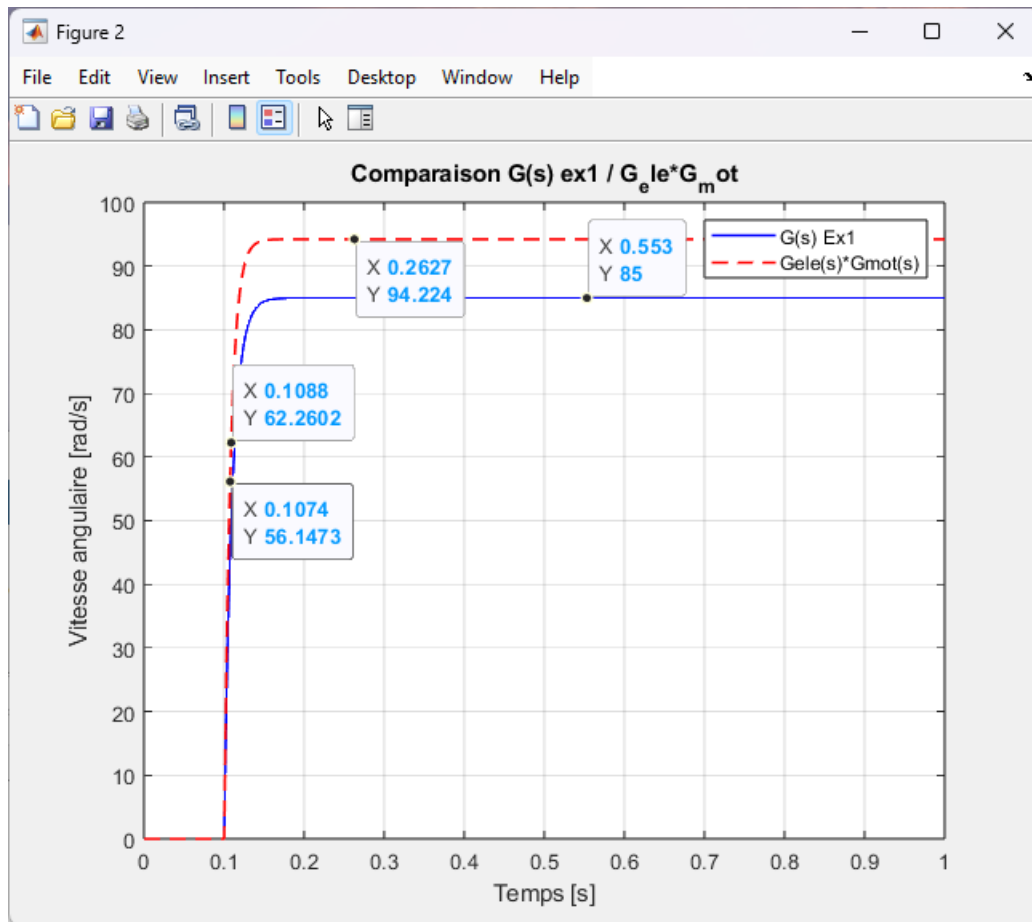
Les fonctions de transfert des sous systemes sont respectivement :

$$G_{mot}(s) = \frac{4.53 \cdot 10^3}{1 + s \cdot 8 \cdot 10^{-3}}$$

$$G_{ele}(s) = \frac{20.8 \cdot 10^{-3}}{1 + s \cdot 1.5 \cdot 10^{-4}}$$

$$G_{mes}(s) = 10.7 \cdot 10^{-3}$$

3.1 A : Verification graphique



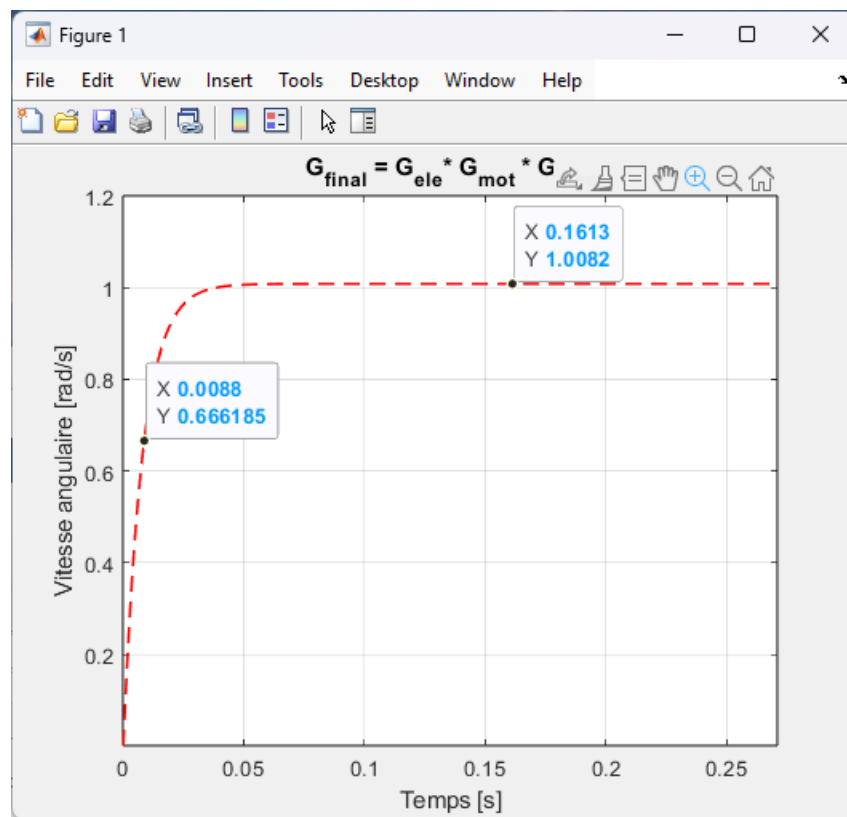
Nous constatons que les deux courbes ne sont pas identiques et que le gain change. Nous constatons également un retard temporel de 0.1s que nous avons ajouté au modèle afin de pouvoir superposer les deux courbes. Elles restent tout de même similaires.

(Comme le titre est illisible, nous pensions à : "Comparaison entre $G(s) / G_{ele} * G_{mot}$ ")

3.2 B : Fonction de transfert du systeme complet

Nous établissons la fonction de transfert à partir de $G_{mot} * G_{ele} * G_{mes}$ tel que :

$$G_{final} = \frac{1.008}{1.2e-06 s^2 + 0.00815 s + 1}$$



(Nous avons compris entre temps comment avoir de jolis titres)

La fonction de transfert de l'ensemble du système G_s (qu'on nomme G_{final} dans le code au point C) est ainsi représentée par le produit de convolution d'un élément de mesure, de l'actionneur et d'un élément sensible.

3.3 C : Code definissant la fonction de transfert

```
load('/MT/systeme/asservis/SIRAu/stepResponse.mat')
t = time;

clear; close all; clc;

s = tf('s');

K_mot = 4.53e3;
tau_mot = 8e-3;
G_mot = K_mot / (1 + tau_mot*s);
K_ele = 20.8e-3;
tau_ele = 1.5e-4;
G_ele = K_ele / (1 + tau_ele*s);
K_mes = 10.7e-3;
G_mes = tf(K_mes);
G_final = G_mot * G_ele * G_mes;
%disp('==== G_s ====')
G_final
```


3.4 D : Application d'un regulateur PI

```

===== G_s =====

G_s =

          1.008
-----
1.2e-06 s^2 + 0.00815 s + 1

Continuous-time transfer function.
Model Properties
===== G_R =====

G_R =

      26.45 s + 3306
-----
              s

Continuous-time transfer function.
Model Properties
===== G_0 avant simplification =====

G_0 =

      26.66 s + 3333
-----
1.2e-06 s^3 + 0.00815 s^2 + s

Continuous-time transfer function.
Model Properties
===== G_0 apres simplification =====

G_0_simpl =

      2.222e07
-----
      s^2 + 6667 s

```

avec :

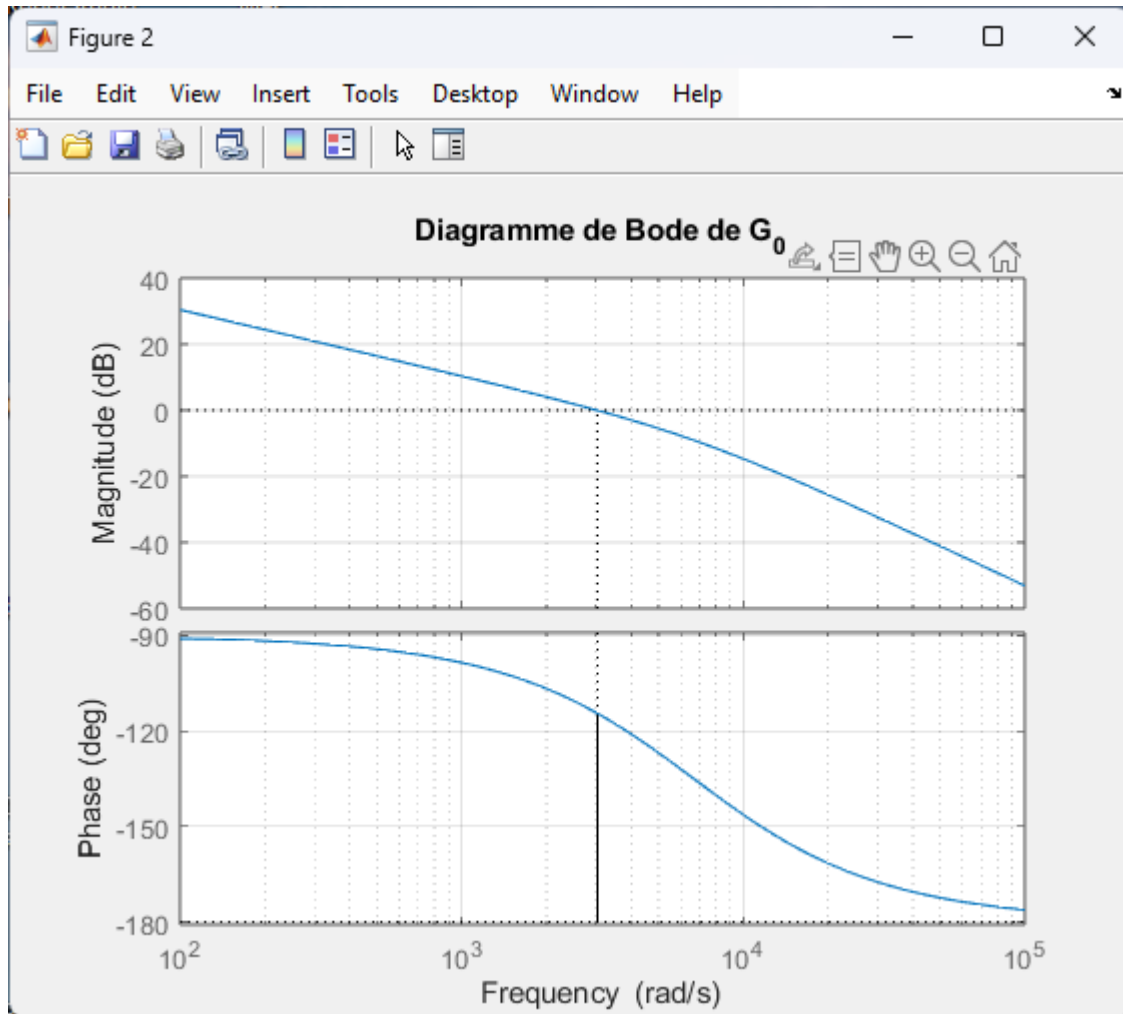
$$G_{cf} = \frac{2.222e07}{s^2 + 6667s + 2.222e07}$$

Voir le code en annexe.

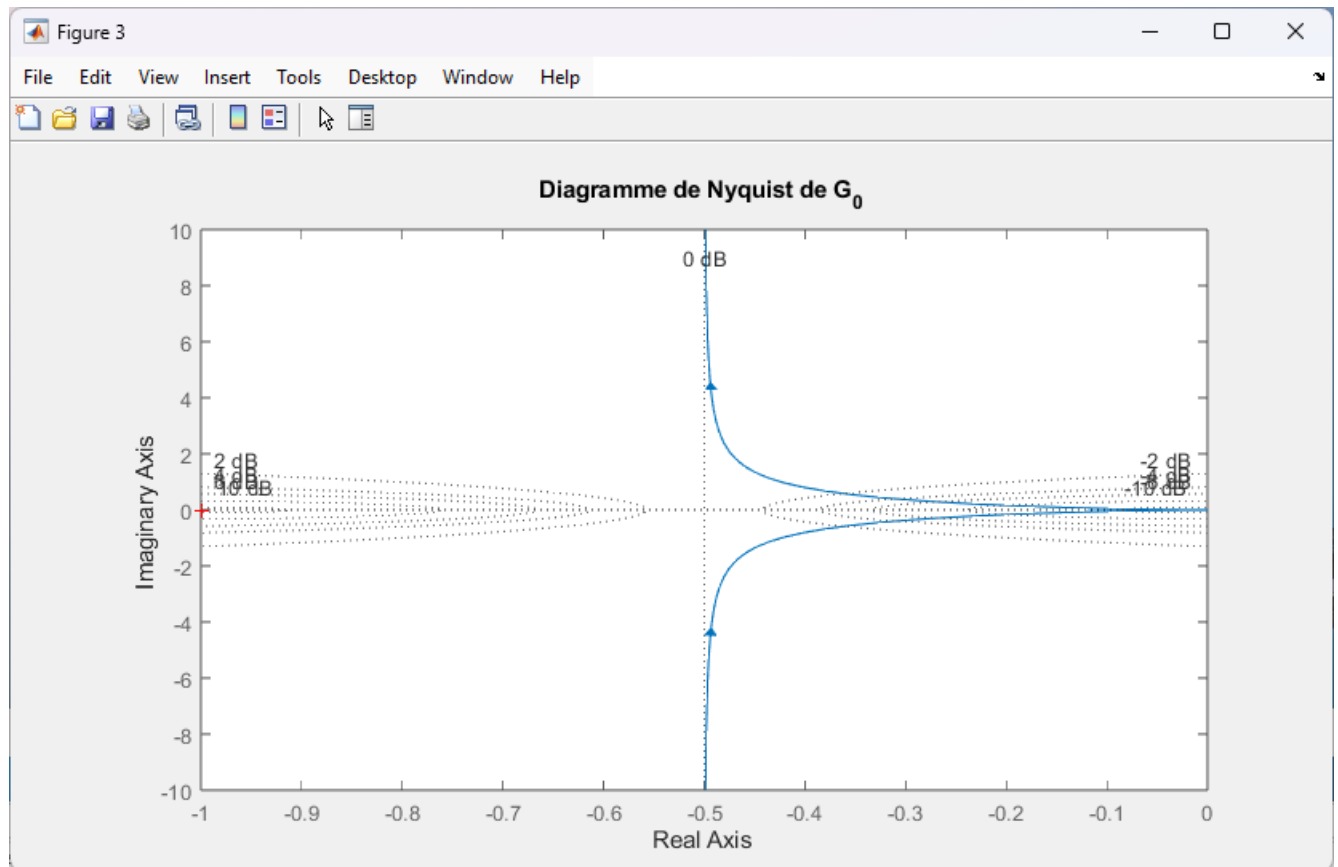
L'application du régulateur transforme le système en boucle fermée (du second ordre). Cela permet de faciliter la suite de l'analyse. La forme est aussi plus simple à lire.

3.5 E : Diagrammes de Bode et Nyquist

La marge de phase est d'environ 65° et la marge de gain est positive. Cela veut dire que le système est stable. BODE :



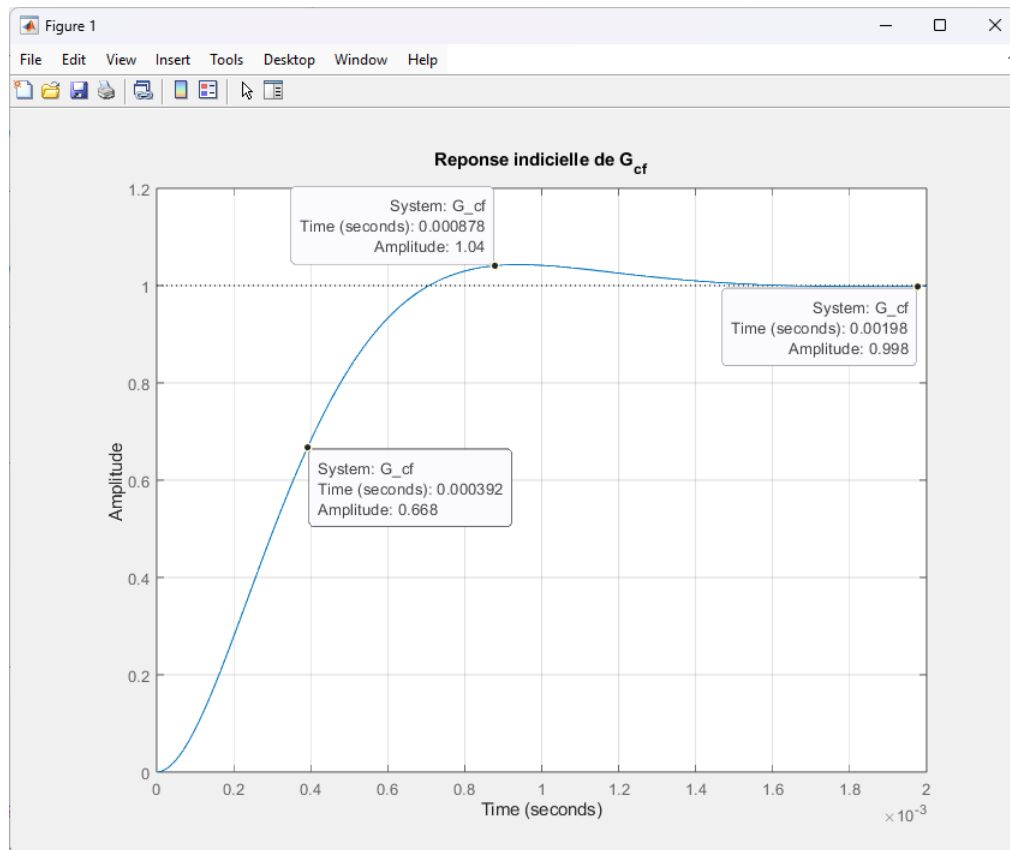
NYQUIST :



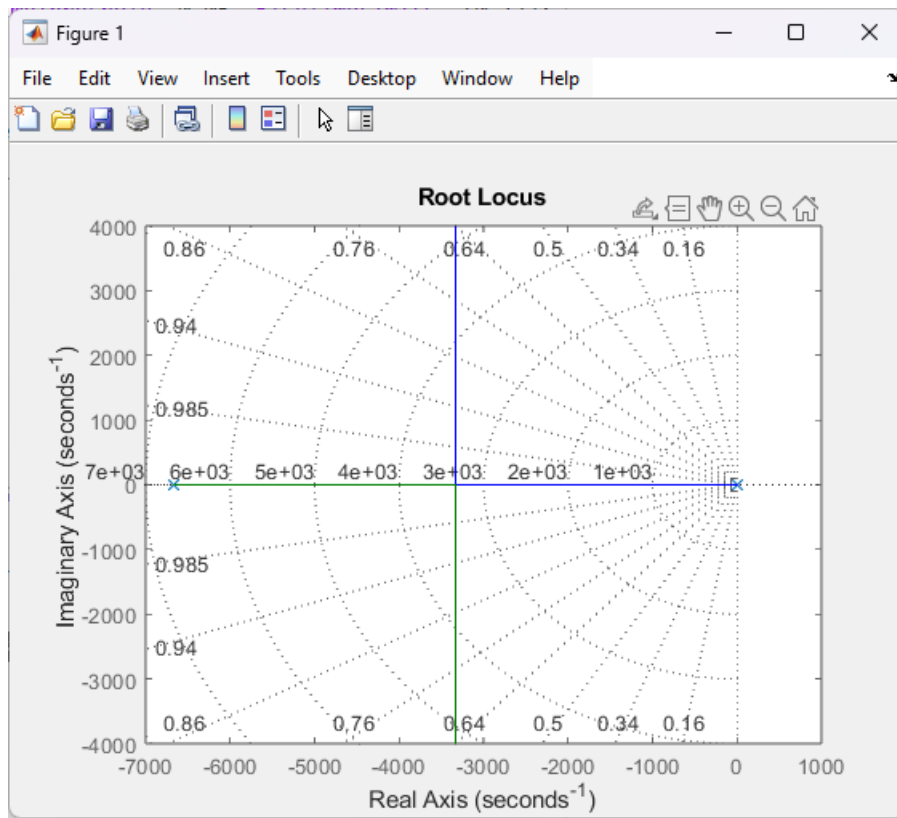
Le diagramme de Nyquist permet de montrer que le lieu de transfert est exclusivement positif. Cela nous permet également d'affirmer la résilience du régulateur PI.

3.6 F : Diagramme d'Evans et reponse indicielle

Reponse indicielle :

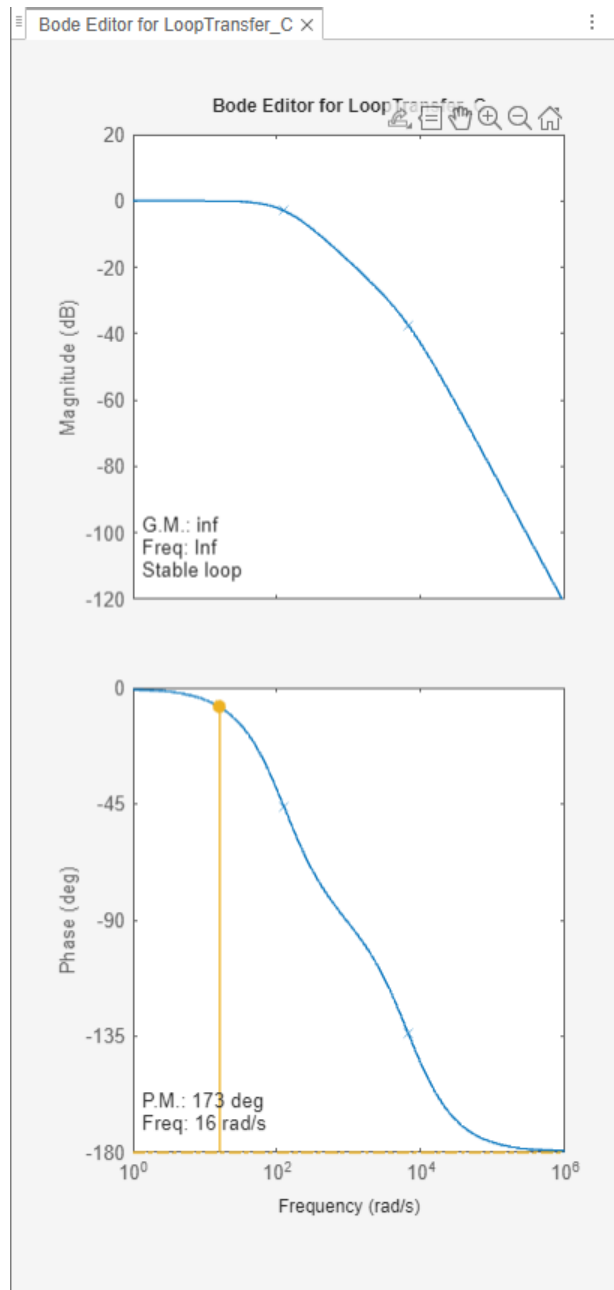


La reponse indicielle illustre un depassement d'environ 5% et un temps de reponse rapide.

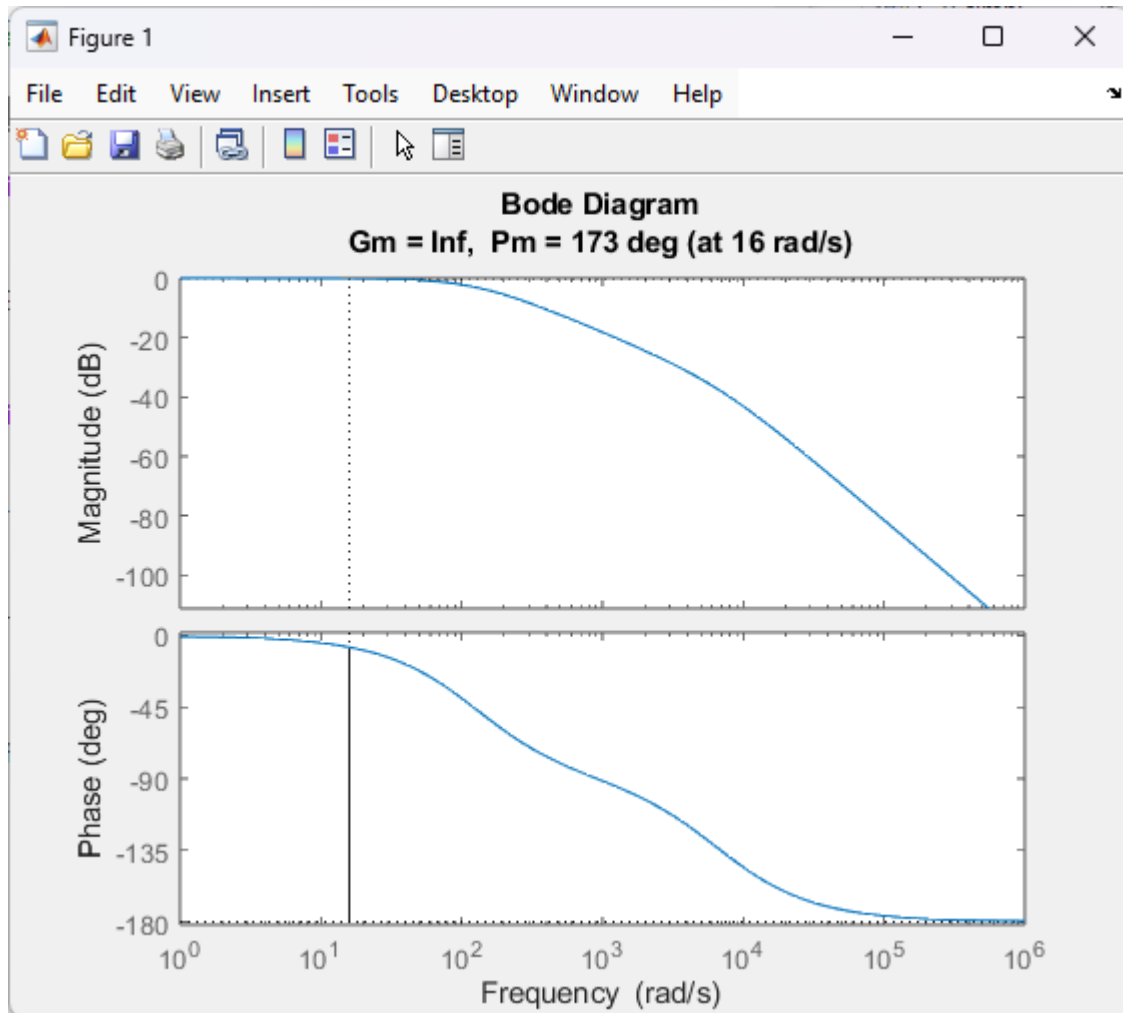


3.7 $G : G(s)$ dans "Control System Designer"

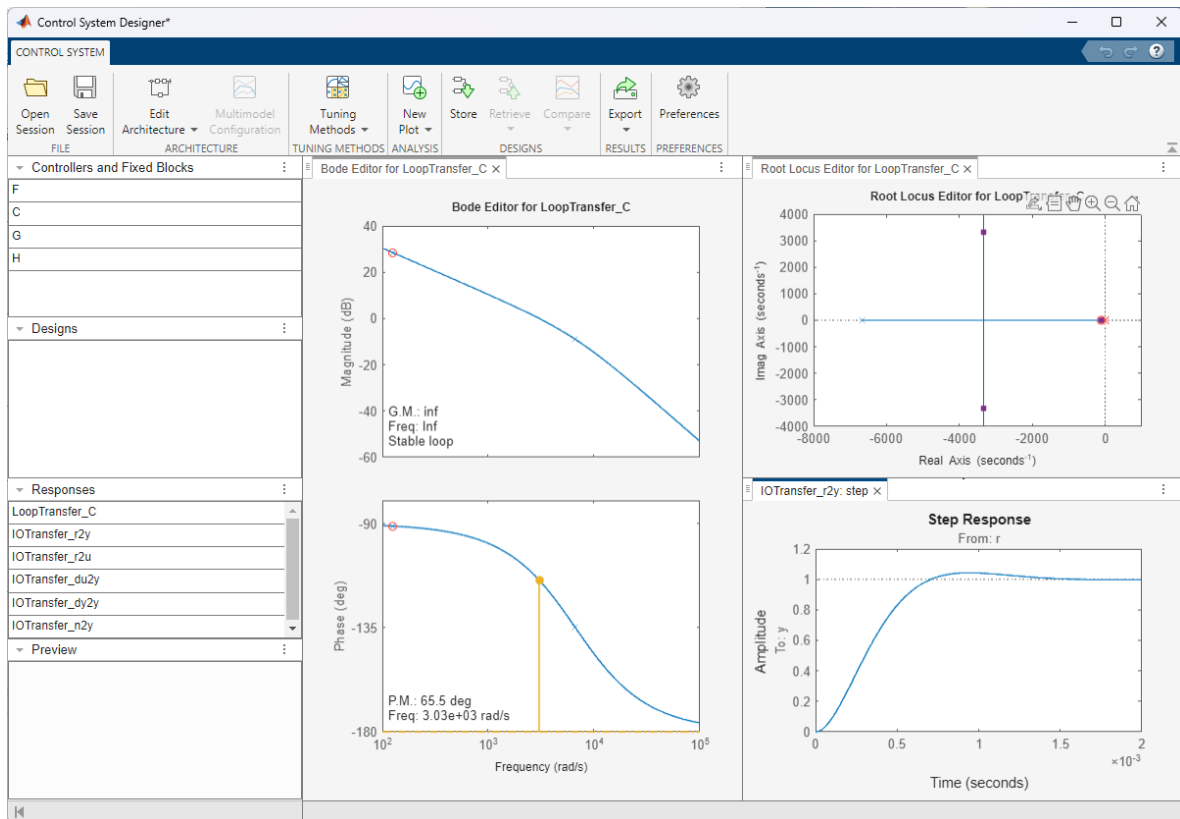
Nous importons G_s dans "Control System Designer" et obtenons :



C'est comparable avec ce que nous obtenons avec la ligne de commande matlab :

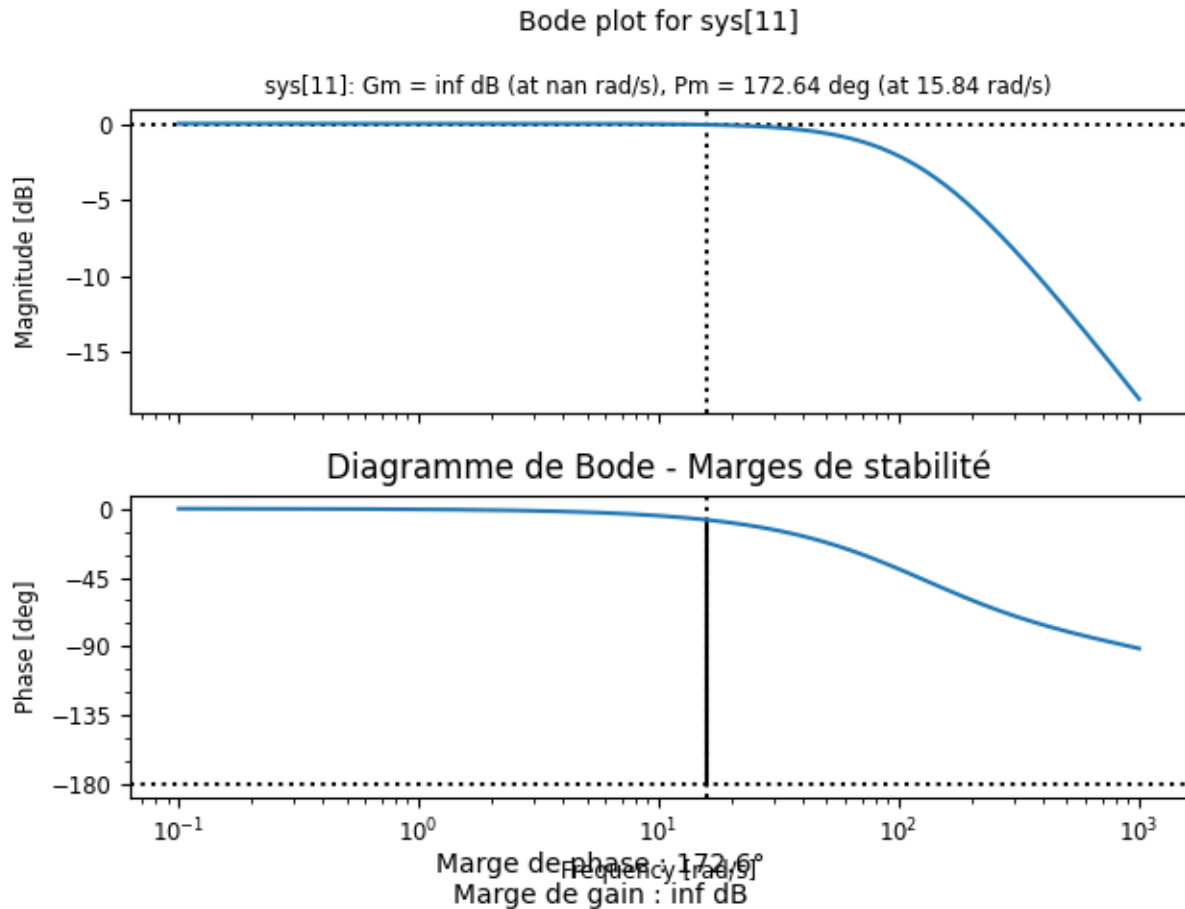


3.8 H : (MATLAB == CSD) ? verification graphique



Nous constatons là encore qu'il s'agit des mêmes schémas qu'avec matlab (voir au point F).

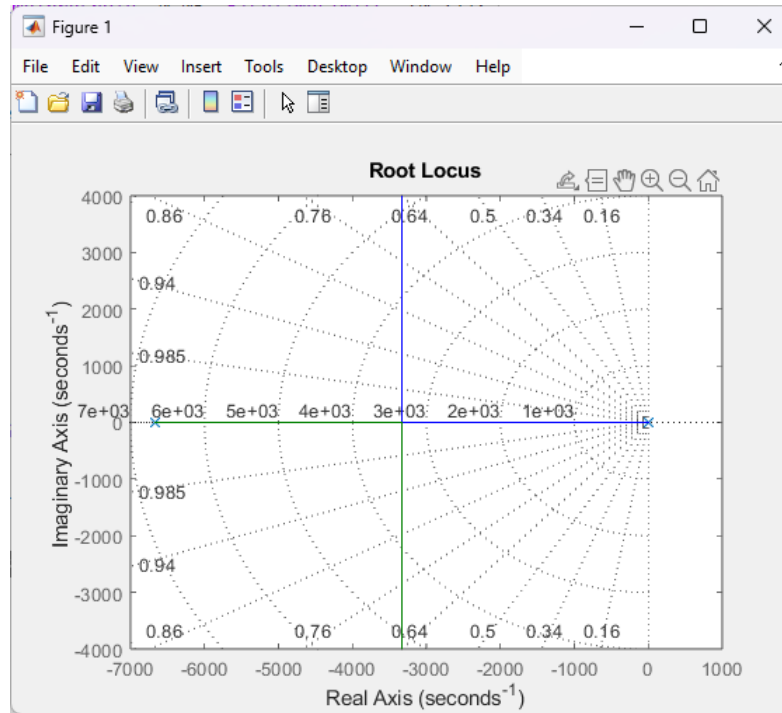
3.9 I : Marges de stabilité sur diagramme de Bode



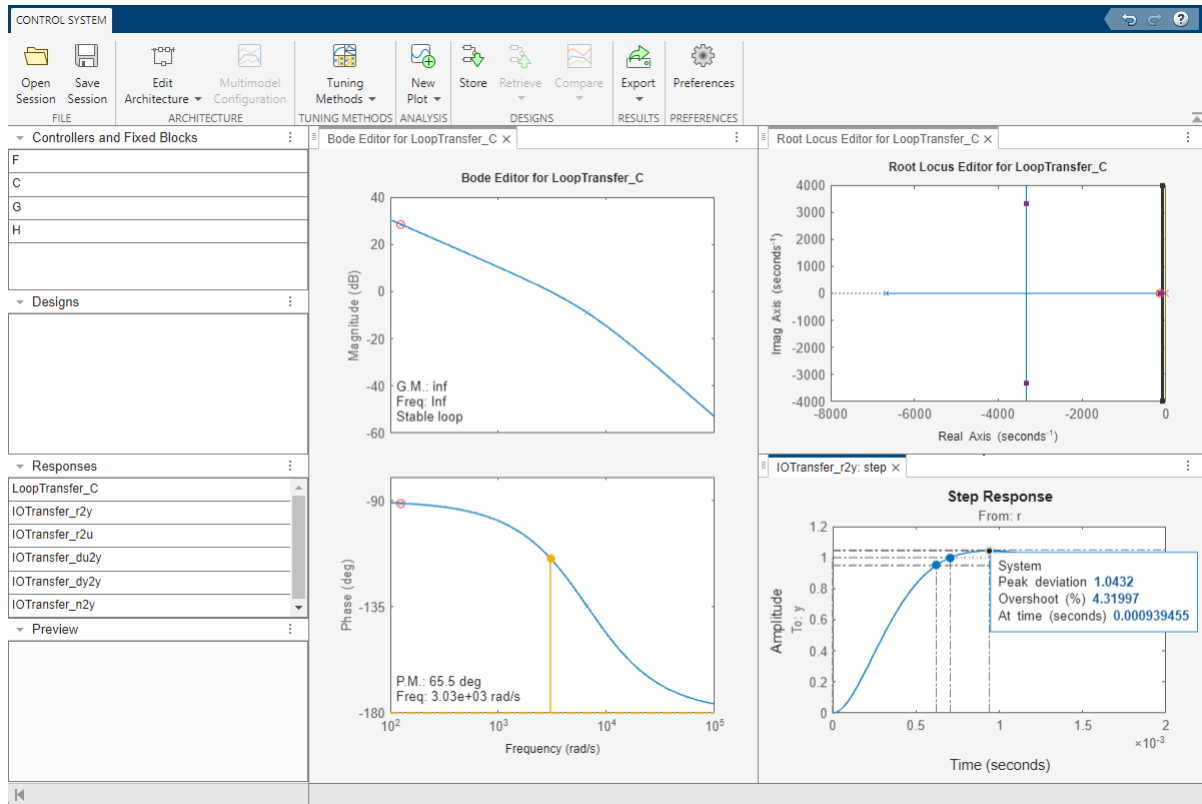
Une marge de phase de 65 et une marge de gain infinie indiquent un système stable et robuste, capable de supporter des perturbations modérées sans devenir instable. Nous n'avons pas eu le temps de le réaliser pendant le laboratoire et avons prit le temps de vérifier comment varient les marges de phase et de gain en fonction de la fréquence. (voir code Python en annexe).

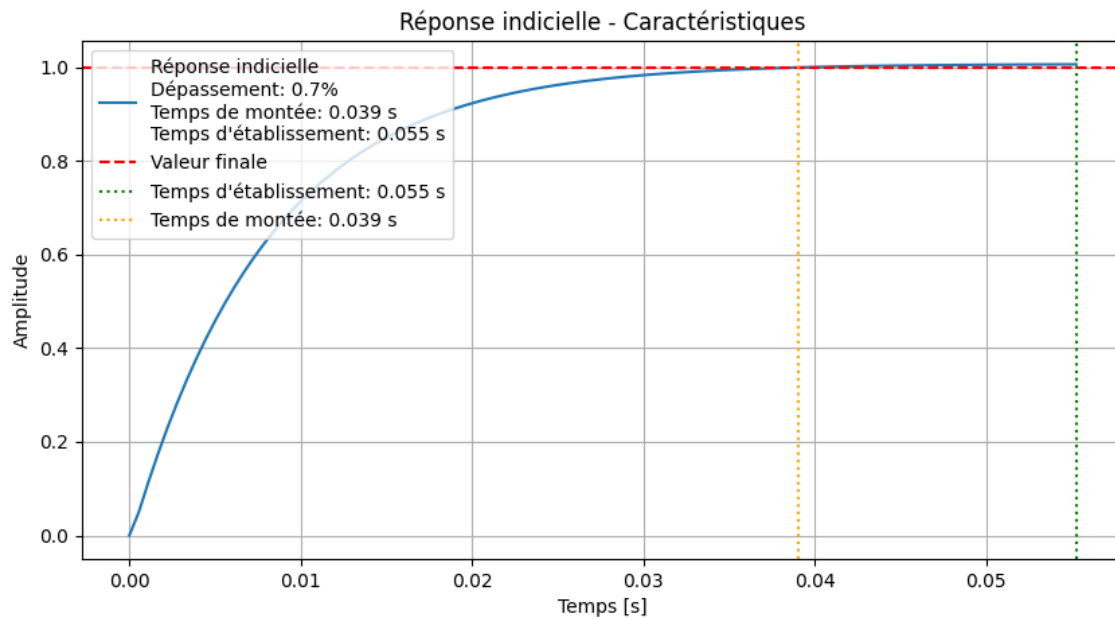
3.10 J : Lieu des poles (vous nous avez dit de passer cette question)

Zeros de Gcf :



3.11 K : Caracteristiques de la reponse indicielle

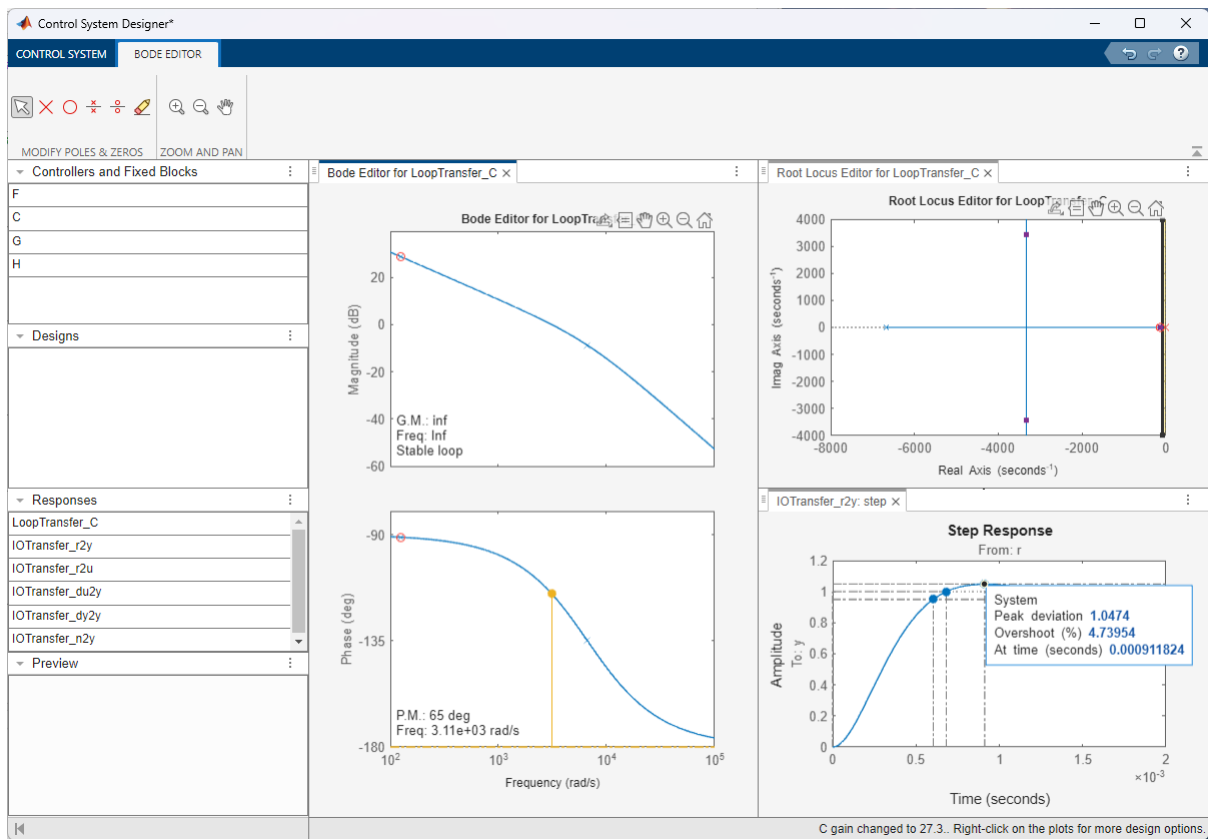




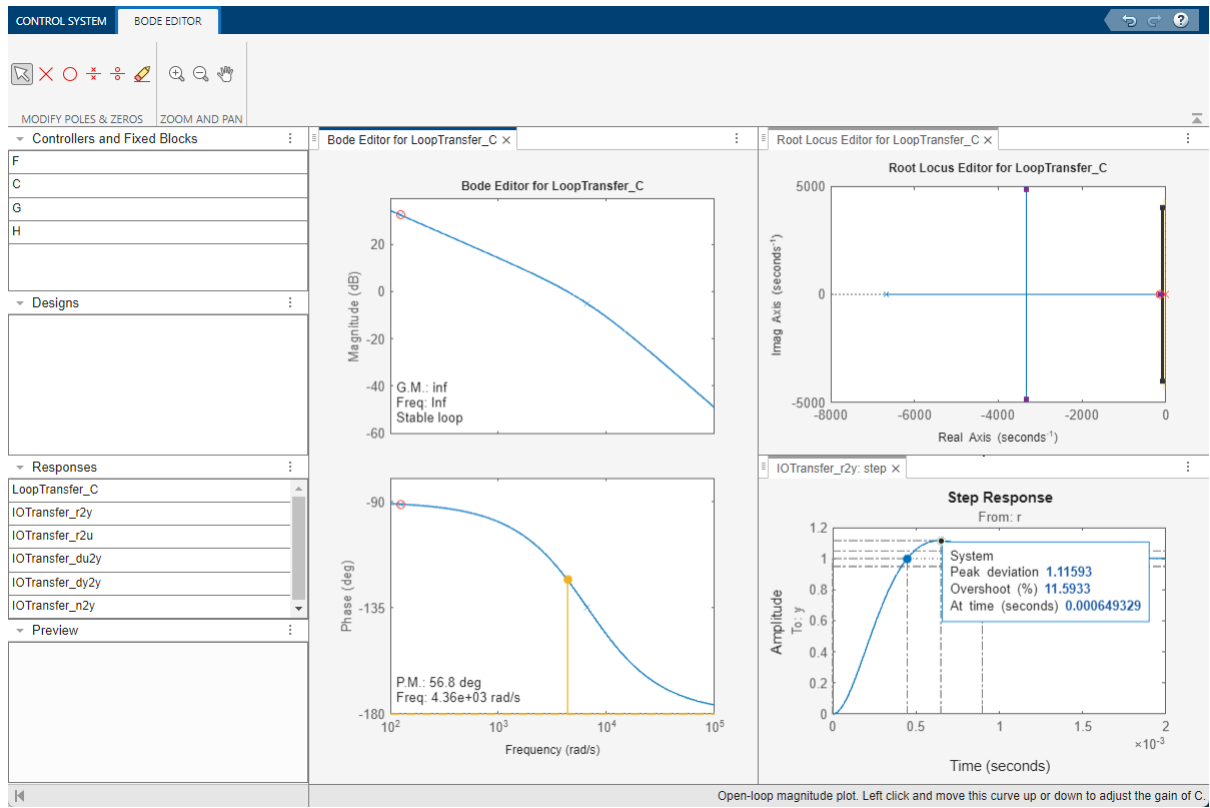
Un dépassement de 5.2%, un temps de montée de 0.047s et un temps d'établissement de 0.21s confirment un bon compromis entre rapidité et stabilité du système en boucle fermée.

3.12 L : Lien entre gain du regulateur et valeurs de depassement

Avec un overshoot de 5% :

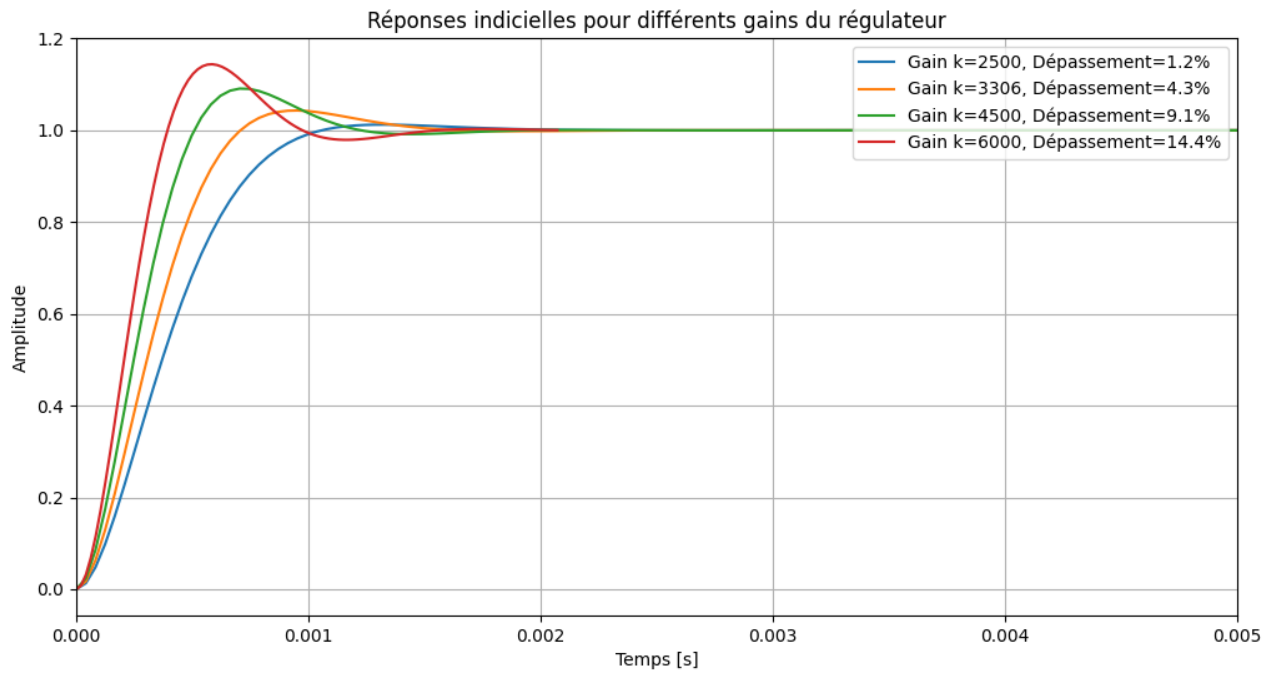


Avec un overshoot de 12% :



Par manque de temps et dans la précipitation, nous n'avons pas cherché les valeurs exactes du contrôle designer, mais nous voyons que le dépassement varie en fonction du gain.

Nous avons pu en un second temps retracer avec python les valeurs de l'énoncé.

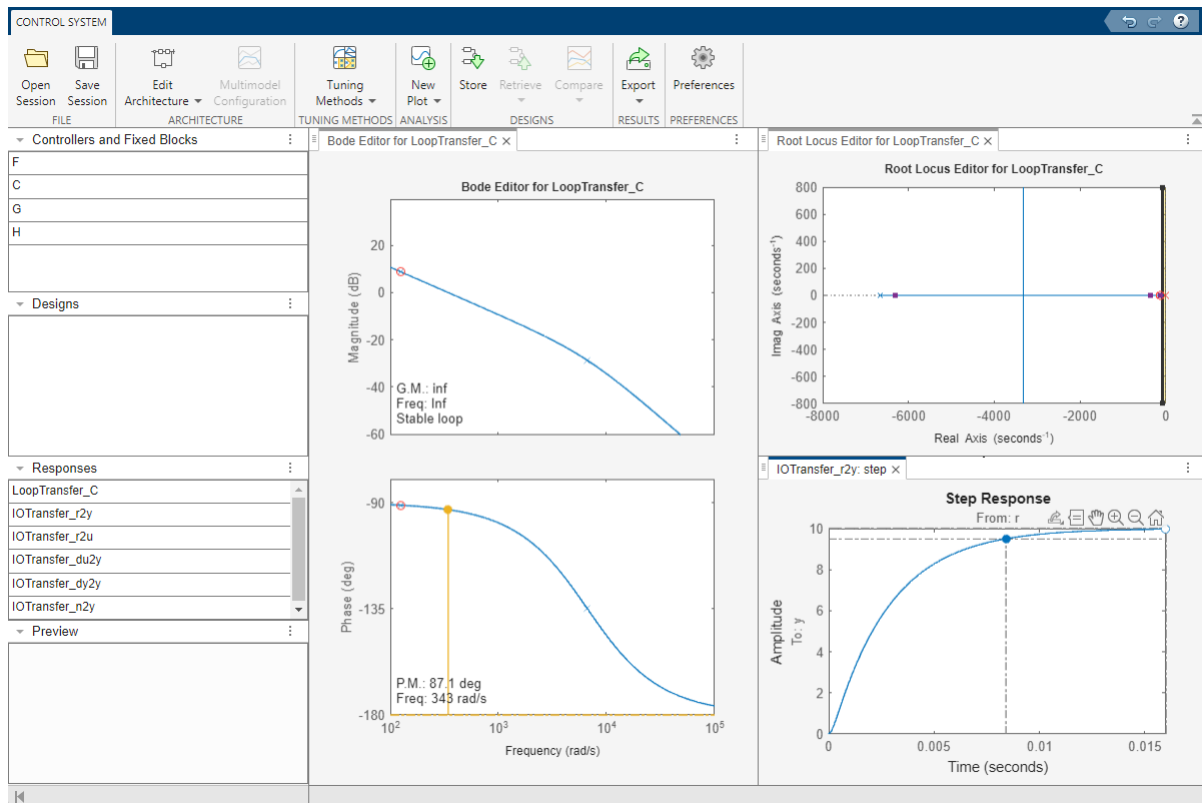


Nous constatons encore qu'en effet la valeur du dépassement (overshoot) est proportionnelle à celle du gain.

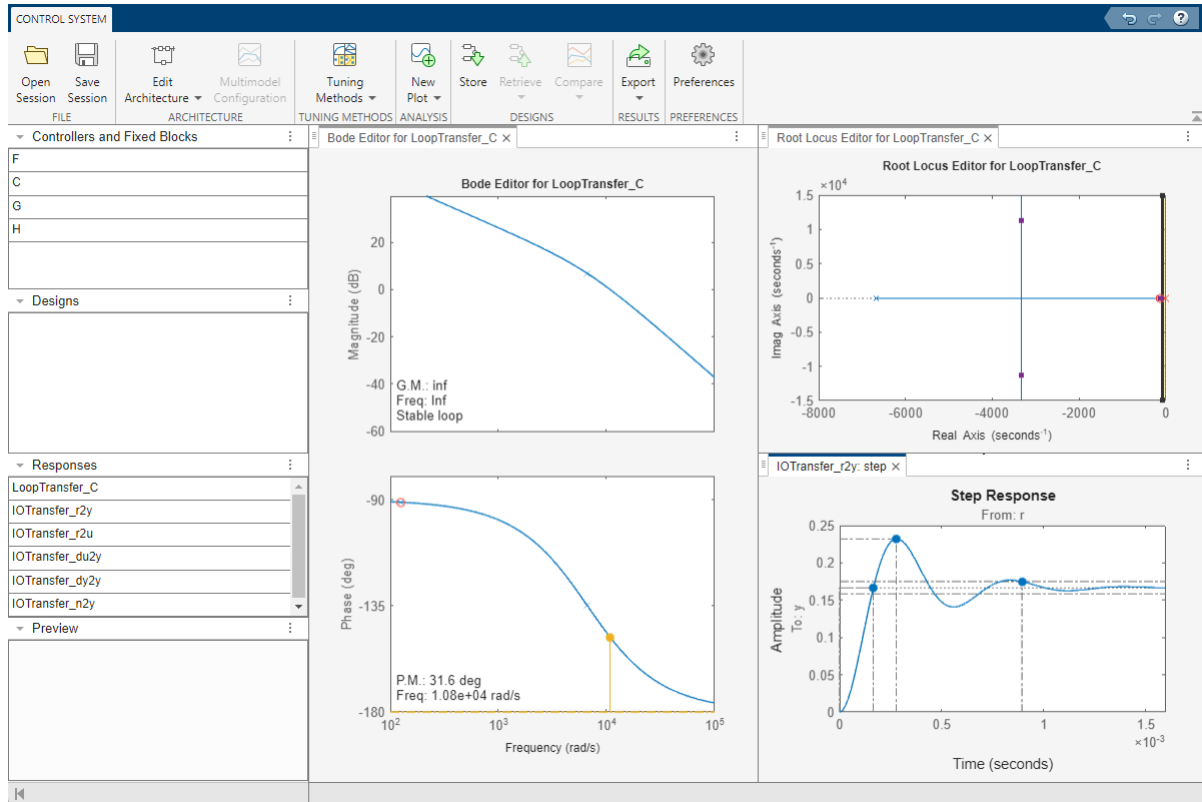
3.13 M : Impact du gain sur la stabilite du systeme

Nous constatons que la modification du gain ne perturbe pas la stabilite du systeme.

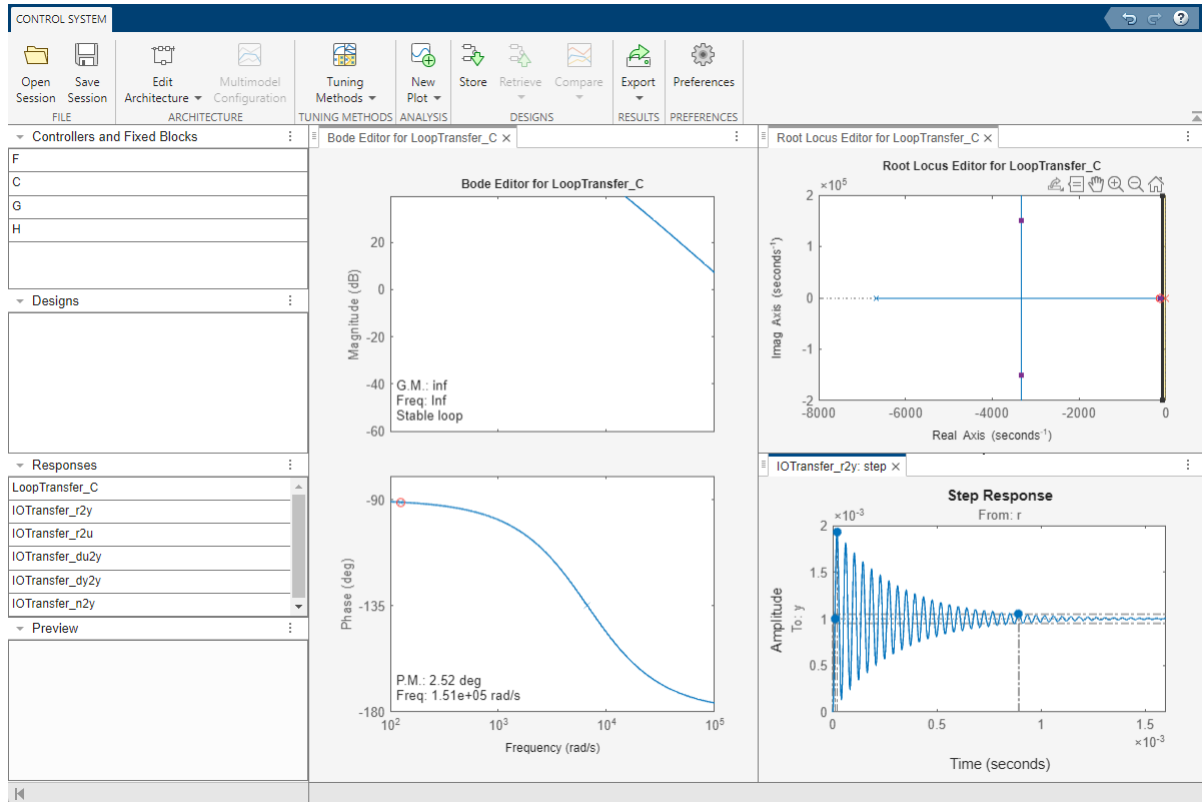
GAIN $k = 0.1$:



GAIN $k = 6$:



GAIN $k = 1000$:



4 Conclusion

Ce laboratoire a permis d'analyser le comportement d'un banc de test SIRAu en reliant les mesures empiriques à une modélisation par fonction de transfert. Après avoir estimé la fonction de transfert du système, un régulateur PI a été appliqué pour obtenir une réponse en boucle fermée stable et performante.

Les diagrammes de Bode, Nyquist et Evans ont confirmé la stabilité du système, avec des marges de phase et de gain positives. Les réponses indicielles ont validé le réglage du régulateur, montrant un dépassement contrôlé et un temps de réponse adapté.

L'utilisation de MATLAB et de Control System Designer a permis de valider les résultats et de comparer les approches théoriques et pratiques. Ce travail a renforcé la compréhension des méthodes d'analyse des systèmes asservis et de leur application concrète

5 annexes (code inexact à cause du compilateur latex)

5.1 Code complet matlab exercice 2

```
load('Z: MT systeme.asservis SIRAu_stepResponse.mat')

t = time;

%% Affichage propre des fonctions de transfert clear; close all; clc;
s = tf('s');

%% G moteur
K_mot = 4.53e3;
tau_mot = 8e-3;
G_mot = K_mot / (1 + tau_mot*s);

%% G element
K_ele = 20.8e-3;
tau_ele = 1.5e-4;
G_ele = K_ele / (1 + tau_ele*s);

%% G mesure
K_mes = 10.7e-3;
G_mes = tf(K_mes);

%% G systeme complet
G_s = G_mot * G_ele * G_mes;

disp('==== G_s ====')
G_s

%% D) Regulateur PI
k_R = 3306;
T_n = 8e-3;

G_R = k_R * (1 + s*T_n) / s;

disp('==== G_R ====')
G_R

%% Boucle ouverte
G_0 = G_R * G_s;

disp('==== G_0 avant simplification ====')
G_0
```

```

%% Simplification
G_0_simpl = minreal(G_0);

disp('===== G_0 apres simplification =====')
G_0_simpl

%% Boucle fermee
G_cf = feedback(G_0_simpl,1);

disp('===== G_cf =====')
G_cf

%% Recuperer numerateur et denominateur
num,den
= tfdata(G_cf,'v');

disp('===== Numerateur de G_cf =====')
num

disp('===== Denominateur de G_cf =====')
den

%% Affichage de la forme Kcf / (a s
2 + b s + c)
K_cf = num(end);
a = den(1);
b = den(2);
c = den(3);

fprintf('
nG_cf(s) = %.6f / (%.6e s
2 + %.6e s + %.6e )
n',K_cf,a,b,c);

%% Verification graphique
figure;
step(G_cf);
grid on;
title('Reponse indicielle de G_cf')

%% Parametres temporels
info = stepinfo(G_cf,'SettlingTimeThreshold',0.05,'RiseTimeLimits',[0 1]);

Overshoot = info.Overshoot

```

```
RiseTime = info.RiseTime
SettlingTime = info.SettlingTime
```

```
%% Bode et Nyquist de la boucle ouverte -j point E)
figure;
margin(G_0_simpl);
grid on;
title('Diagramme de Bode de G_0')
```

```
figure;
nyquist(G_0_simpl);
grid on;
```

```
title('Diagramme de Nyquist de G_0')
```

```
%% EVANS -j Point F
figure;
rlocus(G_0_simpl); % trouver les yeros
grid on;
```

```
figure;
step(G_cf);
```

5.2 Code python point I

```
import numpy as np
import matplotlib.pyplot as plt
import control as ctrl
```

```
Fonction de transfert du système complet (à adapter)
s = ctrl.TransferFunction.s
G_s = 1.008 / (1.2e-6 * s**2 + 0.00815 * s + 1)
```

```
Calcul des marges de stabilité
gm, pm, wg, wp = ctrl.margin(G_s)
```

```
Tracé du diagramme de Bode avec légende détaillée
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
ctrl.bode(G_s, dB=True, Hz=False, omega=np.logspace(-1, 3, 1000), margins=True)
plt.title("Diagramme de Bode - Marges de stabilité")
plt.legend(
    "Gain (dB)",
    "Phase (deg)",
```

```
f'Marge de phase: pm:.1f",
f'Marge de gain: gm:.1f dB"
```

```
,
loc='upper right'
)
```

```
plt.tight_layout()
plt.savefig("_i2.png")
```

```
Affichage des valeurs
print(f'Marge de phase : pm:.1f")
print(f'Marge de gain : gm:.1f dB")
```

5.3 Code python point K

```
Fonction de transfert en boucle fermée (à adapter)
G_cf = ctrl.TransferFunction([1.008], [1.2e-6, 0.00815, 1])
```

```
Réponse indicielle
t, y = ctrl.step_response(G_cf)
```

```
Calcul des caractéristiques
overshoot = np.max(y) - 1 Dépassement en
settling_time = np.where(np.abs(y - 1) < 0.05)[0][-1] * (t[1] - t[0]) Temps d'établissement à
5
rise_time = np.where(y >= 1)[0][0] * (t[1] - t[0]) Temps de montée (0 à 100
```

```
Tracé de la réponse indicielle avec légende détaillée
plt.figure(figsize=(10, 5))
plt.plot(t, y, label=f'Réponse indicielle
nDépassement: overshoot*100:.1f
plt.axhline(1, color='red', linestyle='-', label='Valeur finale')
plt.axvline(settling_time, color='green', linestyle=':', label=f'Temps d
'établissement: settling_time:.3f s')
plt.axvline(rise_time, color='orange', linestyle=':', label=f'Temps de montée: rise_time:.3f
s')
plt.title("Réponse indicielle - Caractéristiques")
plt.xlabel("Temps [s]")
plt.ylabel("Amplitude")
plt.legend(loc='upper left')
plt.grid(True)
plt.savefig("_k2.png")
```

Affichage des valeurs

```
print("Caractéristiques de la réponse indicielle :")
print(f"Dépassement maximal: overshoot*100:.1f%")
print(f"Temps de montée: rise_time:.3f s")
print(f"Temps d'établissement: settling_time:.3f s")
```

5.4 Code python point L

Fonction de transfert du système complet

```
s = ctrl.TransferFunction.s
G_s = 1.008 / (1.2e-6 * s**2 + 0.00815 * s + 1)
```

Gains du régulateur à tester

```
gains = [2500, 3306, 4500, 6000]
overshoots = []
```

```
plt.figure(figsize=(12, 6))
for k in gains:
    Régulateur PI
    G_R = ctrl.TransferFunction([k * 8e-3, k], [1, 0])
    Fonction de transfert en boucle fermée
    G_cf = ctrl.feedback(G_R * G_s, 1)
    Réponse indicielle
    t, y = ctrl.step_response(G_cf)
    overshoot = np.max(y) - 1  Dépassement en
    overshoots.append(overshoot * 100)
    plt.plot(t, y, label=f"Gain k={k}, Dépassement={overshoot*100:.1f}

plt.title("Réponses indicielles pour différents gains du régulateur")
plt.xlabel("Temps [s]")
plt.ylabel("Amplitude")
plt.legend(loc='upper right')
plt.grid(True)
plt.xlim(0, 0.005)
plt.savefig("_l2.png")
```

Tableau des résultats

```
print("Tableau des gains et dépassements :")
print("— Dépassement souhaité — Gain du régulateur — Valeur de dépassement (")
print("—————")
for i, k in enumerate(gains):
    print(f"— '4.3%' if i==0 else '5.0%' if i==1 else '10%' if i==2 else '16.3%' — k — over-
shoots[i]:.1f% —")
```